

图论建模

图论建(jian)模(dan)求(ti)解(xuan)法(jiang)

author:yyandy

2023 年 12 月 25 日

1. 2-SAT
2. 差分约束
3. 最短路
4. 二分图与网络流

2-SAT

SAT 是适定性 (Satisfiability) 问题的简称。一般形式为 k - 适定性问题，简称 k -SAT，但当 $k > 2$ 时该问题为 NP 完全的。

研究这些内容超出了本课范围，因此这里我们只研究 $k = 2$ 的情况。

2-SAT, 简而言之就是要解决这么一类问题: 给出 n 个取值为 $0, 1$ 的变量 x_i , 已知 m 个限制 $\langle a, b, v_a = 0/1, v_b = 0/1 \rangle$, 表示 $x_a = v_a$ 和 $x_b = v_b$ 这两个条件中至少满足一个。

判断能否构造出满足所有 m 个条件的一组变量, 有些时候要求构造一组方案。

对于限制 $\langle a, b, v_a = 0/1, v_b = 0/1 \rangle$ 由于变量只有 0, 1 两种取值, 所以当变量 a 不为 v_a 时, 变量 b 必须为 v_b , 变量 b 不为 v_b 时, 变量 a 必须为 v_a 。

- 将变量的取值看成点, 用点 $p_{i,0}$ 和 $p_{i,1}$ 表示第 i 个变量取 0/1 时的情况。
- 可以看出, 限制的形式大概可以转化为 “第 a 个变量取值为 $\dots$ 可以推出第 b 个变量取值为 $\dots$ ” 的形式。
- 把这些限制用有向边表示, 令一条有向边 (u, v) 表示能够从点 u 的情况推出点 v 的情况。

举个例子: 限制 $\langle 1, 2, 0, 1 \rangle$, 需要连有向边 $p_{1,1} \rightarrow p_{2,1}, p_{2,0} \rightarrow p_{1,0}$ 。

解法 1 (暴力)

- 考虑依次确定变量的取值，对于第 i 个变量，先它取值为 0 的情况，dfs 按照图中的推下去。
- 如果出现不合法，也就是存在 $p_{i,0}, p_{i,1}$ 都被推出，那么就考虑取值为 1 的情况，如果两种情况都不合法，那么就无解。
- 确定完一个变量之后，找到下一个没有被确定的变量重复这个过程。

这个解法时间复杂度是 $O(nm)$ ，但是容易发现我们这样构造出的序列是字典序最小的。

解法 2 (缩点)

缩 SCC，此时 SCC 内的点能够相互推出，如果存在一个 i 使得 $p_{i,0}, p_{i,1}$ 在同一个 SCC 中那么就说明能从第 i 个变量取值为 0 推出第 i 个变量值为 1，第 i 个变量取值为 1 推出第 i 个变量值为 0 也就矛盾了，因此无解。

否则考虑构造一组方案：

对缩点之后的图拓扑排序，对于每个 i 取 $p_{i,0}$ 和 $p_{i,1}$ 中拓扑序较大的那一个，因为这样一定不能推出另外一个，从而保证方案合法。

- 给定 n 个 01 串，每个字符串**至多有一位是未知的**。
- 对于未知的位置可以填 0 或 1。
- 你需要使得任意一个字符串不是其它任意一个字符串的前缀。
- 构造一组方案或者输出无解。
- $1 \leq n \leq 5 \times 10^5$ ，字符串总长不超过 5×10^5 。

- 对于一个字符串至多有一个位置是未知的。
- 因此每一个字符串至多只能有两种状态，未知位置填 0 时的状态我们设为 $\neg x$ ，填 1 的状态为 x 。
- 对于前缀关系，我们常常用字典树进行刻画，我们将所有 $2n$ 个字符串都插入到字典树中去。
- 对于一个字符串 s_x ，它在字典树上的祖先都是它的前缀，需要向它的祖先对应的所有字符串 s_y 连“若存在 s_x 则不存在 s_y ”的边。
- 暴力连会寄，不过既然我们已经建出了字典树，就可以按照字典树连边进行树上优化建图即可。
- 另外需要处理多个字符串相同的特殊情况。

差分约束

差分约束系统是一种特殊的 n 元一次不等式组，它包含 n 个变量 $x_1, x_2, \dots, x_n$ 以及 m 个约束条件，每个约束条件是由两个其中的变量做差构成的，形如 $x_i - x_j \leq c_k$ ，其中 x_i, x_j 为两个变量， c_k 为可正可负的常数。

解法

- 一元一次不等式组可以化为 $x_i \leq x_j + c_k$ 。
- 这与最短路中的三角形不等式 $dis_i \leq dis_j + w_{j,i}$ 十分相似。
- 于是我们可以把每个变量 x_i 看成一个节点，对于限制 $x_i \leq x_j + c_k$ 就从 j 向 i 连一条边权为 c_k 的有向边。
- 将 0 号点设为源点并向每一个点连一条边权为 0 边，跑单源最短路。
- 若图中存在负环（也就是说可以推出 $x_i \leq x_i - C, C > 0$ ），则给定的差分约束系统无解。
- 否则， $x_i = dist_i$ 为该差分约束系统的一组解。

- 由于可能有负权，所以一般使用 Bellman-Ford 或者 SPFA 实现。
- 时间复杂度 $O(nm)$ 。
- 用 SPFA 实现时判断负环要判断一个点是否入队超过 n 次，但是在实际应用中常常使用卡时等技巧。

例题 联合省选 2021 矩阵游戏

- 给定一个 $(n-1) \times (m-1)$ 的矩阵 $B_{i,j}$, 其中 $B_{i,j} = A_{i,j} + A_{i,j+1} + A_{i+1,j} + A_{i+1,j+1}$ 。
- 你要构造出一个满足条件的 $n \times m$ 的矩阵 $A_{i,j}$ 且 $0 \leq A_{i,j} \leq 10^6$, 或者判断无解。
- 多组数据, $T \leq 10, n, m \leq 300, B_{i,j} \leq 4 \times 10^6$ 。

- 如果不考虑 a 的值域限制，那就可以钦定 a 的第一行第一列都为 0 这样依次推出所有的数，设此时推出的这个不一定满足条件的矩阵为 a' 。
- 发现给 a 的一行或者一列作 $+1, -1, +1, \dots$ 这样的修改之后生成的 b 恰好不变。
- 利用这个性质，我们可以进行调整，设 r_i 表示第 i 行进行了多少次这样的操作，列同理，设为 c_j 。
- 那么经过调整后第 i 行第 j 列的值就是 $a_{i,j} = (-1)^i \times c_j + (-1)^j \times r_i + a'_{i,j}$ ，也就要满足 $0 \leq a_{i,j} \leq 10^6$ 。
- 这样就可以转化为 $2nm$ 个形如 $r_i \pm c_j \leq A$ 或 $r_i \pm c_j \geq B$ 的限制条件。

- 如果只存在 $r_i - c_j \leq A$ 或 $\geq B$, 那么就是差分约束的板子, 存在有加法。
- 对于 i 为奇数的 r_i 将 r_i 的定义改为 “对第 i 行进行了 r_i 次 $-1, +1, -1, \dots$ 的操作”。
- 对于 i 为偶数的 c_i 将 c_i 的定义改为 “对第 i 列进行了 c_i 次 $-1, +1, -1, \dots$ 的操作”。
- 此时所有的条件都被转化为 $r_i - c_j \leq A$ 或者 $c_i - r_j \leq B$ 的形式, 直接差分约束即可。
- 使用 SPFA 理论时间复杂度 $O(Tn^3)$, 但是实际跑的很快, 可以通过。

例题 01 Balanced

- 构造一个长度为 n 的 01 序列。
- 有 m 条限制 l_i, r_i 表示 l_i, r_i 之内的 0 和 1 个数相等。
- 你需要构造出所有合法的 01 序列中字典序最小的。
- $1 \leq n \leq 10^6, 0 \leq m \leq 2 \times 10^5$ 。

- 很容易想到对 01 序列作前缀和 S_i 然后得到 $S_i \leq S_{i+1}$, $S_i + 1 \geq S_{i+1}$ 和 $S_{l_i-1} + \frac{r_i-l_i+1}{2} = S_{r_i}$ 这三种限制。
- 直接用 SPFA 跑差分约束就可得到字典序最大解，简单处理就可以得到字典序最小解。
- 发现上面建图的主要问题就是在解决 l_i, r_i 限制的时候会连负权边。
- 我们可以换一种方式建图，设 pre_i 表示 $1 \sim i$ 中 0 个数 - 1 个数。
- 这样 l_i, r_i 的限制相当于连 0 边，这样就只有 0, 1 的边了。
- 用 01 BFS 跑差分约束即可得到字典序最小解（因为我们构造出的 pre_i 字典序最大，也就是 0-1 最多）。

最短路

最短路树和最短路 DAG

- 在做以 s 为源的最短路的过程中，对于每个点记录它是由哪一个点拓展到的，设为 f_i 。
- 对于每个节点 i 连接 $f_i \rightarrow i$ ，这样就形成了一棵叶向树，也就是最短路树。
- 如果对于将 $i \rightarrow j$ 且 $dis_i + w_{i,j} = dis_j$ 的每对 (i, j) 都加入图中，这样就会形成一个 DAG，称为最短路 DAG。
- 最短路树和最短路 DAG 在求解单源最短路径问题的变形时发挥了很大的作用。（比如说之前讲过的 K 短路）

- 给定 n 点 m 边的**正权无向图**，和 q 次询问。
- 每次询问给定 e, y 表示将编号为 e 的边边权改成 y ，要求回答修改后 1 到 n 之间的最短路。
- **询问之间独立，也就是修改只对当前询问起作用。**
- $1 \leq n, m, q \leq 2 \times 10^5$ 。

- 最短路树的经典应用。
- 将路径分为经过询问中边的 e 和不经过的两种，设这两类路径最短的长度分别是 D_1, D_2 。
- 则询问的答案为 $\min(D_1 - val_e + y, D_2)$ ，而其中 $D_1 - val_e + y$ 容易求，所以我们需要解决的就是求解 D_2 ，也就是删除 e 之后的最短路。
- 对于 e 不在最短路上的情况，删去显然并不影响最短路长度。
- 否则先跑出以 1 为源点的最短路树 T_1 和以 n 为源点的最短路树 T_n ，将问题放在最短路树上考虑。

删边最短路 Indecisive Taxi Fee

- 由于是无向图，显然一定存在一条原图的最短路径 Q 使得先走 T_1 的树边，然后再走 e 然后再走 T_n 的树边到达 n 。
- 设 $id(u)$ 为节点 u 在 Q 上的下标， $pre(u)$ 为 u 和 n 在最短路径树 T_1 上的 LCA， $suf(u)$ 表示 u 和 1 在最短路径树的 LCA。
- 对于一条删边后的最短路径 P ，它走 T_1 的树边，然后再走一段非树边，然后再走 T_n 的树边。
- 设 e 连接的两点为 (p_i, p_{i+1}) ，那么可以找到 P 中第一个满足 $id(suf(u)) \geq id(p_{i+1})$ 的点 u 以及它在 P 上的前驱 v 。
- 则 $id(pre(v)) \leq id(suf(v)) \leq id(p_i)$ ，因此可以选择路径 $1 \rightarrow pre_v \rightarrow v$ 变得不劣，对于 $suf(u)$ 同理。
- 所以对于所有的路径，我们都能够调整为 $1 \rightarrow pre(v) \rightarrow v \rightarrow u \rightarrow suf(u) \rightarrow n$ ，这样就只有 $v \rightarrow u$ 一条边不在树上。

- 得到了删边之后的最短路最多只有一条边不在最短路树上的结论之后就好做了。
- 对于每一条不在最短路树上的边 $u \rightarrow v$, 都找到 $pre(u)$ 和 $suf(v)$, 这样就可以计算出这条路径比原来的最短路长度的增加量。
- 删掉最短路上的边 $x \rightarrow y$ 时, 相当于对经过的非树边有一个 $id(pre(u)) \leq id(x), id(suf(v)) \geq id(y)$ 的限制。
- 这个问题可以离线下来扫一遍或者直接使用数据结构维护。
- 时间复杂度是单 $\log$ 的。

同余最短路用于求解在某个范围内有多少重量可以由若干物品做**完全背包**凑出。或者说，更加具体地，给出 $n, n \leq 50$ 个数 a_i ($1 \leq a_i \leq 10^5$)，求 $[L, R]$ ($1 \leq L \leq R \leq 10^{18}$) 以内满足能表示成 $\sum a_i c_i, (c_i \geq 0)$ 的正整数个数。 L, R 的值域往往非常大，因此朴素的背包就不能适用了。

解法 1

- 尝试将其转化为图论问题, 从这 n 个数中随意找到一个“基准” a_r , 那么如果 x 能够被表示出, 那么 $x + ka_r (k \geq 0)$ 也可以被表示出。
- 于是自然地, $\text{mod } a_r$ 进行考虑, 设 f_i 表示最小的 x 使得 x 能够被表示出且 $x \bmod a_r = i$ 。
- 对于所有的 $0 \leq i < a_r$, 和所有的 $1 \leq j \leq n$ 都连出从 i 到 $(i + a_j) \bmod a_r$ 边权为 a_j 的一条边。
- 从 0 开始跑最短路求出每一个点的 f_i , 最后统计答案部分是容易的。
- 由于图有一些性质, 所以 SPFA 跑得非常快, 复杂度不会分析。
- 其实也可以转化成 01BFS 来做, 时间复杂度为 $O(na_r)$ 。

解法 2

- 这个解法和图论没什么关系，但是还是提一嘴。
- 借鉴最短路做法之中的一些思想，选择一个基准物品 r 考虑 $\text{mod } a_r$ 下的完全背包。
- 对于所有的 a_i ，它能够在长度为 a_r 的这一个环上形成 $\text{gcd}(a_i, a_r)$ 个子环。
- 不可能对于一个子环，绕着环走很多圈，否则可以用基准物品代替其中的绕圈。
- 因此，最多往背包里加入 $\frac{a_r}{\text{gcd}(a_i, a_r)} - 1$ 个 a_i ，也就是说我们绕着每一个子环转两圈就能够保证环上每一个点都能够转移到环上的任意点。

二分图与网络流

二分图相关性质

- **性质 1; 二分图最小点覆盖 = 二分图最大匹配**
- 证明：首先通过最大匹配构造出一组合法的点覆盖方案。
- 按照匈牙利算法找增广路的过程，对于跑完最大匹配后的图，从未被匹配的做部点开始，先走一条非匹配边，然后再走一条匹配边，将所有这样的路径上的点打上标记。
- 对于这样的一条路径，从右部点到左部点走的是匹配边，左部点到右部点走的是非匹配边。最终点覆盖集合里的点就是左部点中未打标记的点和右部点中打了标记的点。
- 对于每条非匹配边，它的两 endpoint 都会被标记。而对于一条匹配边，由于图上不存在增广路，所以这样的路径一定是从左部点开始在左部点结束，因此匹配边要么两端都标记了，要么都没标记，也就是匹配边被覆盖了。
- 上面关于匹配边的说明已经表明一条匹配边只有**恰好**一个端点被选入点覆盖集合，也就是选出的点数等于最大匹配，由此我们通过构造证明了二分图最小点覆盖 $\leq$ 二分图最大匹配。
- 而显然，每一条匹配边都需要选两端至少一个点因此二分图最小点覆盖 $\geq$ 二分图最大匹配，于是证毕。

二分图相关性质

- **性质 2: 二分图最大独立集 = n - 二分图最大匹配**
- 独立集和点覆盖一一对应, 取不在最小点覆盖中的所有点就能够满足条件。
- 根据点覆盖定义不会存在一条边两端都不在集合里, 因此不在点覆盖里的点的集合两两之间没边。
- **性质 3: 二分图最大团 = n - 补图最大匹配**
- 因为团要求两两之间都有连边, 对于一个点集, 一旦在补图上有连边, 那么就一定在原图上没有连满边。
- 因此就是二分图最大团就是补图最大独立集。

DAG 最小路径划分

- 问题描述：给定 DAG 找出若干条不交路径，使得每个点都在恰好一条路径上，求最少选多少路径。
- 我们很难直接表示一个点是否被覆盖，考虑拆点。
- 可以将点拆成入点 in_x 和出点 out_x 并且如果原图有一条 $u \rightarrow v$ 的边，那么就连接 out_u 和 in_v 。
- 对这张图跑二分图最大匹配 f ，最小路径覆盖就是 $n - f$ ，匹配边组成的路径就是方案。
- 因为每匹配一对相当于是将图上的“链”减少 1。

- 问题描述：给定 DAG 找出若干条可交路径，使得每个点都在至少一条路径上，求最少选多少路径。
- 考虑传递闭包之后转化为上一个问题求解。
- 传递闭包之后，每个点相当于能够“跳过”一些中间的节点。

- 问题描述：给定一个 DAG，找出一个最大的点集，使得点集里面的点两两不可到达。
- **根据 Dilworth 定理：DAG 最长反链大小 = 最小路径覆盖。**
- 实际上 Dilworth 定理描述的是偏序集最长反链 = 最小路径划分。
- 将 DAG 传递闭包之后反链不影响，而 DAG 变为了偏序集，对这个传递闭包之后的
- Dilworth 应该是上一节课的内容，这里就不证明了。
- 而比较显然，在偏序集上最大独立集就是最长反链。

- **最基本的最小割最大流定理：最大流等于最小割**
- 最大流建模常常把流当成移动, 路径或者选择。
- 最小割建模常常将割掉边当成删除元素或者是划分集合。
- 对于删掉元素而不是删掉边的最小割, 我们常常使用拆点的技巧, 即将一个点拆成 in_x 和 out_x 并连边, 割掉这一条边就相当于删掉元素 x 。

集合划分模型

- 题目描述：将 n 个元素划分为两个集合 S_1, S_2 ，最小化代价。
- 代价分为 $\sum_{u \in S_1} \sum_{v \in S_2} c_{u,v} + \sum_{u \in S_1} a_u + \sum_{u \in S_2} b_u$ 。
- 大概就是将一个元素划分进两个集合分别有不同的代价，然后有若干对元素之间有额外代价。
- 对于这一类问题，可以考虑将元素 i 与 S 和 T 连边，流量为分进 S_1/S_2 的代价，然后 $u \in S_1, v \in S_2$ 时会产生贡献的 u, v 连流量 $c_{u,v}$ 。
- 需要特别注意的是，当 u, v 双向都能产生代价的时候需要连双向边。
- 这样跑最小割，相当于在割出来的图中与 S 相连的部分形成的是 S_1 ，其它部分形成的是 S_2 。

最大权闭合子图

- 定义：对于一张**有向图**的闭合子图为一个点集 V' 满足点集里面的任意一个点 x 能够到达的点都 $\in V'$ ，最大权闭合子图同时还要满足点权和最大。
- 可以应用上一张中的集合划分模型，此处可以将 S_1 和 S_2 作为选/不选的集合。
- 将点 u 划分进 S_1 的代价是 w_u ，划分进 S_2 的代价是 0，如果有边 $u \rightarrow v$ ，那么隔开 u, v 的代价是 $-\infty$ 。
- 但是直接做要跑最大割，为了解决这个一种想法是首先将所有正权点都加到集合中。
- 这样相当于将一些负权点加入集合或者将一些正权点移除集合，求最少得代价满足闭合子图性质。
- 具体地，对于 $w_u > 0$ 连 $S \rightarrow u$ 流量为 w_u 的边，否则连 $u \rightarrow T$ 流量为 $-w_u$ 的边，最终答案就是 $\sum_{w_u > 0} w_u - \text{mincut}(S, T)$ 。

“串联”模型 切糕

- 有 n 个 $[1, m]$ 中的数 $a_1 a_2, \dots, a_n$ 。
- 将 a_i 赋值为 j 则需要付出 $v_{i,j}$ 的代价。
- 有 q 个限制, 每个限制形如 $a_x - a_y < z$ 或 $a_x - a_y > z$ 。

“串联”模型 切糕

- 将一个数 a_i 拆成 $m + 1$ 个点 $A_{i,j}$, 然后从 $A_{i,j}$ 向 $A_{i,j+1}$ 连流量为 $v_{i,j}$ 的边, 割掉这一条边就表示 $a_i = j$ 。
- 然后需要连接 $A_{i,j+1} \rightarrow A_{i,j}$ 流量为 $+\infty$ 的边 (为了传递限制)。
- 对于一个 $a_x - a_y < z$ 的条件, 从所有 $A_{x,i}$ 向所有 $A_{y,i-z}$ 连流量为 $+\infty$ 的边, 这样就可以保证 $a_x - a_y < z$ 。

- 显然最小费用流每流一单位流量，费用单调不降。因此当我们不要求最大化流量时当单位流量费用为正的时候停止就可以最小化费用了。
- 解决一类带有凸性的问题：一些题目中会出现“一条边每流一单位流量就有对应的费用”，如果费用单调不降可以将边拆成若干流量为 1 的边处理。

区间选择模型

- 给定 $[1, m]$ 中的 n 个区间 $[l_i, r_i]$ ，你可以选择每个区间若干次进行区间覆盖，每个区间选择一次的代价为 w_i ，最多可以选 c_i 次。
- 要求使得任意点 j 被覆盖次数在 $[a_j, b_j]$ 间，求最小/最大代价。
- 将 1 到 $m+1$ 连成一条链，尝试用 $i \rightarrow i+1$ 边的流量刻画 i 被覆盖的次数。
- 对于一个区间，我们建立从 l_i 到 r_i+1 的一条边，称为区间边，如果区间边上流过一条流，就表示选择一次这个区间。
- 源流向 1, $m+1$ 流向汇，首先假设确定总流量为 X ，那么没有流经 $i \rightarrow i+1$ 这条边的流量就是选择了跨过 i 的区间，所以 $i \rightarrow i+1$ 的流量为 $X-f_i$ 也就是表示 i 被覆盖了 f_i 次。
- 只需要设定 $i \rightarrow i+1$ 的流量上下界为 $[X-b_i, X-a_i]$ ，然后将区间边费用设为区间代价，容量设为可选次数，最小/最大费用上下界最大流即为答案。一般将 X 设定为一个足够大的值。

例题 Making It Bipartite

- 有 n 个点, 点有点权 a_i 。
- 对于 (i, j) 如果满足 $a_i | a_j$ 那么就在 i 和 j 之间连一条无向边。
- 现在希望删去一些点, 使图变成二分图, 求出最少删去点的数量。
- $n, a_i \leq 5 \times 10^4$

题解 Making It Bipartite

- 当存在长度为 3 的链的时候，建出来的图就不是二分图了。
- 于是一个点要么只有入度，要么只有出度，拆点为 in_x 和 out_x 。
- 对于原图上的一条边 $u \rightarrow v$ ，相当于是 $out_u \rightarrow in_v$ 。
- 直接做最少删去节点不好做，考虑最多保留多少节点。
- 于是我们连接不合法的边也就是 $in_u \rightarrow in_v$, $out_u \rightarrow out_v$, $in_u \rightarrow out_v$ 和 $in_u \rightarrow out_u$, $in_v \rightarrow out_v$ 然后求这个图的最大独立集。
- 原图是偏序集，而发现这样连出的边仍然是偏序集，跑最长反链即可。

例题 Showing Off

- 给定一个 $n \times m$ 的矩阵 S 。
- 你需要构造两个 $n \times m$ 矩阵 A, B ，其中权值矩阵 A 只包含**正整数**， B 矩阵是一个只包含 L/R/D/U 的字符矩阵或者判断无解。
- 构造出的矩阵需要满足一下条件：以每一个 (i, j) 为起点根据 $B_{i,j}$ 的值，往上 (U) 下 (D) 左 (L) 右 (R) 四个方向移动无限次，设这样经过的格子的集合为 $P_{i,j}$ 。
- 你需要使得从 (i, j) 出发行走过程中能够到达的格子 $(x, y) \in P_{i,j}$ 的权值 $A_{x,y}$ 的和为 $S_{i,j}$ 。
- 此外你还需要保证从任意位置都不能够走出格子外面。
- $1 \leq n \times m \leq 10^5$ $2 \leq S_{i,j} \leq 10^9$

- 首先考虑按照 B 矩阵连边，那么最终的这个图一定是基环树森林。
- 对于基环树环上的节点，它们的 S 权值一定相等。
- **由于这个基环树森林是在网格上构建的，所以无论怎样，都不存在是奇环。**
- 根据上面的关键性质，如果一个基环树的环包含了两个以上的格子，那么我们可以把环上的点两两匹配，使得大环被拆成多个二元环。
- 容易发现，所有连成更大的大环的情况都可以被二元环所替代。

- 因此，我们考虑匹配，一个点可以和它周围 S 相同的点进行匹配。
- 由于是在网格上，将网格黑白染色之后可以发现这是二分图匹配。
- 现在的问题就是如何判断哪一些点在二元环上。
- 简单分类讨论一下：
- 对于有 S 更小的邻居的格子，我们可以直接向那个更小的邻居连边，而必须放到环上（但也可能在环上）。
- 对于周围 S 都大于它的格子，无法连边，无解。
- 否则周围有等于它的格子且没有比它小的格子，那么当前这个格子一定要在环上，否则就没地方连了。

- 我们将问题转化为二分图匹配问题，其中有一些点必须找到匹配，一些点可匹配可不匹配。
- 我们可以通过上下界网络流来解决这个限制，从源点向“必环点”连流量 $(1, 1)$ 的边，向“可环点”连流量 $(0, 1)$ 的边。
- 跑一遍有源汇上下界可行流就可以了。
- 时间复杂度 $O(nm\sqrt{nm})$ 。
- 构造方案根据匹配的方案构造就行了，细节有一点多。

例题 危桥

- 有 N 个岛屿，有 M 个双向桥梁连接，每个桥梁有两种类型，一种可以走无限次，一种只能够走 2 次。
- 有两个人，一个人要在 S_1, T_1 之间往返 P_1 次，一个人要在 S_2, T_2 之间往返 P_2 次。
- 问是否存在一种行走方案能够满足条件。
- $1 \leq N \leq 50$ ，保证无重边，自环。

- 由于是无向图，可以将一个人行走次数翻倍去掉往返的条件。接下来如果只有一个人行走就是典型的最大流了，但是问题是存在两个人。
- 比较容易想到的乱搞：设置源点为 S_1, S_2 ，汇点为 T_1, T_2 跑最大流（给 S_1, T_1 限流 $2P_1, S_2, T_2$ 限流 $2P_2$ ）。
- 但是显然这样是伪的，因为可能会有 $S_1 \rightarrow T_2, S_2 \rightarrow T_1$ 这样混在一起的流。
- 但是只需要第二次设置源点为 S_1, T_2 ，汇点为 S_2, T_1 再进行最大流，如果还是满的，那么就确定有解。

- 证明：第一次最大流中如果满流，因为 S_1, T_1, S_2, T_2 点的流量被限制，所以 $S_1 \rightarrow T_2, S_2 \rightarrow T_1$ 流量相等，设为 x ，那么 $S_1 \rightarrow T_1$ 流量为 $P_1 - x$ ， $S_2 \rightarrow T_2$ 的流量为 $P_2 - x$ 。
- 对于第二次最大流，同理可设 y 。
- 以 $S_1 \rightarrow T_1$ 为例，不妨设 $x \leq y$ ，那么可以构造出 $S_1 \xrightarrow{x} T_2 \xrightarrow{y} T_1$ 这样一个流量为 x 的流，再加上第一次 $S_1 \rightarrow T_1$ 的流，就能形成合法方案。

例题 Snuke the Phantom Thief

- 在二维平面上，有 n 颗珠宝，第 i 颗珠宝在 (x_i, y_i) 的位置，价值为 v_i 。
- 现在有一个盗贼想要偷这些珠宝，有 m 个偷珠宝的限制，限制有以下四种：
 - 横坐标小于等于 a_i 的珠宝最多偷 b_i 颗。
 - 横坐标大于等于 a_i 的珠宝最多偷 b_i 颗。
 - 纵坐标小于等于 a_i 的珠宝最多偷 b_i 颗。
 - 纵坐标大于等于 a_i 的珠宝最多偷 b_i 颗。
- 这四个限制输入的时候分别用 LRDU 四个字母来区分。
- 问在满足这些约束的条件下，盗贼能偷的珠宝的最大价值和是多少。
- $n \leq 80, 1 \leq x_i, y_i \leq 100, 1 \leq m \leq 320$

题解 Snuke the Phantom Thief

- 给出限制比较令人难受，没有办法直接建图。
- 将限制进行转化，比如对于 x 坐标小于等于 a_i 不能选择超过 b_i 这个限制，相当于是“所有偷走的珠宝中，（从小到大排序后）排名大于 b_i 的 x 坐标大于 a_i ”。
- 由于还有“横坐标大于等于 a_i 的珠宝最多偷 b_i 颗。”这一类限制，我们需要枚举偷掉珠宝的总数，将其转化为“排名小于 b_i 的 x 坐标小于 a_i ”。
- 于是这样就将关于坐标的限制转化为了关于排名的限制，也就是“偷走的宝石的 x/y 坐标中（从小到大排序后）排名为 i 的宝石的 x/y 坐标落在区间 $[l_i, r_i]$ 内”。

题解 Snuke the Phantom Thief

- 转化为排名的限制之后就可以直接建图。
- 将排名抽象为点，相当于给每个排名选择一个对应的珠宝（珠宝的坐标要在排名的坐标范围内）。
- 由源点连出 $(1, 0)$ 的边，连向 x 坐标排名点。
- 从 x 坐标排名的点向珠宝的入点连 $(1, 0)$ 的边。
- 然后珠宝的出入点之间连 $(1, v_i)$ 的边。
- 珠宝的出点向 y 坐标点连 $(1, 0)$ 的边。
- y 坐标点向汇点连 $(1, 0)$ 的边。
- 跑一个最大费用最大流，如果没法满流，说明偷不了这么多珠宝，否则把答案对费用 `checkmax`。
- 这样流一个单位的流量，就相当于“串”起了 x 坐标排名、珠宝、 y 坐标排名，显然正确。
- 由于还要枚举选择的数量，时间复杂度要再乘个 n ，但是能过。

- 可能是比较 useless 的东西？
- 与一般网络流相比，你需要对于所有点对 (x, y) 回答它们之间的最小割的值。
- 需要比暴力对于所有点对跑网络流更加优秀的做法。
- 我们可以建出一棵树，满足树上两点的最小割的值等于原图上两点的最小割的值，然后就相当于查询树上两点之间的边权最小值。
- 其实这不是严格的最小割树（最小割树能够求方案），而是“等价流”树。
- 接下来的内容是证明为什么能够建出这样一棵树，以及怎么建出这样一棵树。

最小割树

- 建出“等价流”树常用的是 Gusfield 算法，它通过分治来构造这样的一棵树。
- 具体做法就是每次从当前点集中随意取出两个点 x, y 作为源点和汇点。（初始时点集包含所有点）
- 跑出这两个点之间的最小割（注意，要对全图跑最小割，而不是对点集里的点跑最小割）之后，给最小割树上的 x, y 连上一条权值为最小割的边。
- 跑完图被分成了两半，所以我们的点集也被分成了两半，继续对两半递归下去即可。
- 直到当前点集只包含一个节点的时候停止，时间复杂度为 $O(n \times flow)$ ，考虑证明正确性。

最小割树

- **结论 1:** 任意取三个点 a, b, c , 有
$$\text{mincut}(a, c) \geq \min(\text{mincut}(a, b), \text{mincut}(b, c))$$
- 证明: 很显然, 在点 a, c 的最小割中, 点 b 要么和 a 在一起, 要么和 c 在一起。
- 反证: 如果不满足上式的话可以 $\text{mincut}(a, b), \text{mincut}(b, c)$ 至少有一个能够通过取 a, b 之间的最小割来使得取值更小。
- 或者考虑使得 a 和 b, c 都不连通的最小割是 α , b 和 a, c 的为 β , c 和 a, b 的为 γ 。
- 则原式可以化为 $\min(\alpha, \gamma) \geq \min(\min(\alpha, \beta), \min(\beta, \gamma))$, 容易发现成立。
- 由此, 我们还发现 $\text{mincut}(a, c), \text{mincut}(a, b), \text{mincut}(b, c)$ 中至少有两者相等, 且另外一个数大于等于这两个数。

- 有了上面这个结论我们推广得到对于 k 个点, 依然满足 $\text{mincut}(s_1, s_k) \geq \min_{i=1}^{k-1} \text{mincut}(s_i, s_{i+1})$ 。
- **结论 2:** 假设 a, b 的最小割将点集分为 S_a 和 S_b 两个部分, 那么如果 $x \in S_a, y \in S_b$ 那么 $\text{mincut}(a, b) \geq \text{mincut}(x, y)$ 。
- 证明: 显然, a, b 的最小割也是 x, y 的一组合法的割。
- 设最小割树上 u, v 两点路径上最小边权的边是 x, y , 由结论 1 的推广可以得到 $\text{mincut}(a, b) \geq \text{mincut}(x, y)$ 。
- 由结论 2 可得 $\text{mincut}(a, b) \leq \text{mincut}(x, y)$, 因此 $\text{mincut}(a, b) = \text{mincut}(x, y)$ 。
- 所以“等价流”树分治构造的正确性得到了证明, 而且我们可以得到图上两点最小割只有 $O(n)$ 种。

例题 Pumping Stations

- n 个点 m 条边的带权无向图。
- 你要构造一个长为 n 的排列 p , 排列的代价是 $\sum_{i=2}^n \text{mincut}(p_{i-1}, p_i)$ 。
- 其中 $\text{mincut}(S, T)$ 表示图中 S 为源点, T 为汇点的最小割。
- 求最大的收益, 并输出方案。
- $n \leq 200, m \leq 1000$ 。

例题 Pumping Stations

- 首先建出最小割树，然后找到最小割树上最小的那一条边（边权为 d ），这一条边将树分成 X, Y 两个部分。
- 由于我们找到的是最小的那一条边，所以说对于 $a, b \in X$, $\text{mincut}(a, b) \geq d$, 对于 $a \in X, b \in Y$, $\text{mincut}(a, b) = d$ 。
- 由此可以推出我们只会通过这最小的边一次，否则可以调整顺序得到不劣的解。
- 对于 X, Y 进行递归构造就做完了。

【必做】

选做：

[NERC2018] Harder Satisfiability

[省选联考 2021 A 卷] 矩阵游戏

Indecisive Taxi Fee

切糕

危桥

[NOI2012] 美食节

[NOI2008] 志愿者招募

Showing Off

Pumping Stations

GL & HF